

Agentic AI, LLMOps, Cloud Deployment and Privacy

Project checklist audit for Bob IDE

A code-evidenced assessment of what is implemented, what can be added, and what is not needed for the current project scope.

Assessment basis

Repository inspected: code_editor_ai (Bob IDE). Source checklist: 20-page Module 10 assignment PDF. Assessment date: 6 August 2026. No checkbox is marked present from documentation alone when the repository contradicts it; partial controls are explained explicitly.

Status key

PRESENT - directly evidenced in the inspected project. ADDABLE - absent or partial, but relevant and feasible. NOT NEEDED - not applicable to the current scope; reconsider if scope changes.

Executive assessment

Bob IDE is a credible supervised agentic coding prototype. Its strongest controls are the explicit planner/replanner/reviewer stages, persistent workflow state, typed MCP tools, path containment, bounded timeouts, Git integration and a proposal cache that prevents model output from immediately modifying project files. Its main production gaps are formal agent evaluation, retries, complete traces, token/cost/latency telemetry, authentication and authorization, privacy controls, managed secrets, containerization and continuous monitoring.

PRESENT	ADDABLE	NOT NEEDED	TOTAL REVIEWED
40	72	20	132

Highest-priority additions

1. Enforce authentication and server-side authorization before any non-local or multi-user deployment.
2. Create a fixed agent evaluation dataset with task-success, tool-accuracy, safety and regression gates.
3. Add end-to-end trace IDs, structured redacted logs, latency/token/cost telemetry and monitored health checks.
4. Move tokens out of local JSON into a secret store; add PII/retention/encryption rules for context sent to external models.
5. Containerize the gateway, MCP service and frontend; define CI/CD, rollback and production deployment architecture.

Verification note

The repository test command was attempted with the bundled Python runtime. Ten tests completed, but the Python suite stopped during cleanup because Windows reported a test workspace directory as in use. Node, contract, lint and build stages therefore did not run in that attempt. This is reported as a verification limitation, not as proof of a product failure.

Checklist status by section

Each original checklist item is classified below. Evidence paths refer to the inspected repository. 'Present' may include a scope caveat where the implementation is suitable for a local prototype but not a hosted production service.

1. Agentic AI Foundations

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
Has a planner	PRESENT	A dedicated planning stage creates a selectable plan before coding. bob_core/model_service.py:75-115
Has at least two tools	PRESENT	FastMCP exposes a large registry of file, terminal, Git, worktree, model and context tools. mcp_server.py:24-40; capabilities.py
Memory	PRESENT	Plans, runs, model stages and proposals persist across workflow steps in per-workspace JSON stores. This is workflow memory, not long-term user profiling. bob_core/model_service.py:93-110, 185-221; .bob stores described in README
Retry	ADDABLE	Timeouts and fallbacks exist, but model/tool calls do not use a controlled retry policy with limits, backoff and idempotency checks. Add bounded retry in bob_core/colab_adapter.py around transient HTTP failures.
Reflection	PRESENT	Replanning and a distinct reviewer stage inspect intermediate plans/code and return PASS or FAIL. bob_core/model_service.py:118-157, 192-229
Human approval	PRESENT	Generated edits remain proposals until a user applies them; failed/conflicting proposals require an explicit force-confirmation in the UI. bob_core/proposal_store.py:208-234; frontend/src/components/EditorArea.jsx:110-116
Structured output	PRESENT	MCP is configured for JSON responses and plan/model results are normalized into predictable dictionaries. mcp_server.py:24-35; bob_core/colab_adapter.py:13-45
Error handling	PRESENT	Invalid inputs, HTTP failures, timeouts and background-run exceptions are converted to controlled errors or failed run records. bob_core/colab_adapter.py:99-143; bob_core/model_service.py:282-284
Logging	PRESENT	Run status and planner/coder/reviewer artifacts are persisted for inspection. This is useful audit history, though not centralized production logging. bob_core/model_service.py:93, 185, 206, 262-266

Metric coverage: Agent metrics from the source checklist are not calculated. Add a labeled agent test set and record tool decisions, steps, loops, task outcomes and tool-call outcomes.

2. LangChain, LangGraph and CrewAI

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
Tool abstraction	PRESENT	Capabilities have stable names and typed Python signatures and are registered as MCP tools. capabilities.py; mcp_server.py:35
Prompt templates	ADDABLE	The local repository stores user prompts and endpoint contracts, but versioned reusable model prompt templates live outside the inspected project or are absent. Add prompts/ with identifiers, variables, changelog and tests.
State management	PRESENT	Runs, plan selection, stages and proposal states are persisted and linked by project, run and plan IDs. bob_core/model_service.py; bob_core/plan_store.py; bob_core/proposal_store.py
Retry	ADDABLE	Nodes can be manually rerun/replanned, but transient failures are not automatically retried under a policy. Add retry metadata to run records and adapter calls.
Conditional routing	PRESENT	The workflow routes through replan, review PASS/FAIL, conflict handling and stream-to-sync fallback paths. bob_core/colab_adapter.py:228-263; bob_core/proposal_store.py:208-257
Human node	PRESENT	Users select plans and explicitly apply, discard or force proposals. README.md staged workflow; frontend/src/components/SourceControlPanel.jsx
Parallel execution	PRESENT	Independent compatibility runs execute on daemon threads and the gateway supports asynchronous realtime activity. bob_core/model_service.py:234-245; node-server/server.js
Multi-agent design	PRESENT	Planner, coder and reviewer have distinct responsibilities and persisted handoff artifacts. bob_core/model_service.py:75-229

Metric coverage: Workflow completion, handoff accuracy, node success/latency and idle time are not aggregated. Persist node start/end timestamps and expected handoff schemas before calculating them.

3. Practical Agent Integration

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
Every tool documented	ADDABLE	The capability catalog and many docstrings exist, but documentation is not complete enough to explain every tool's purpose and usage conditions. mcp_server.py:38-41; capabilities.py
Input schema	PRESENT	FastMCP derives schemas from typed function signatures and server-side validation rejects unsafe or invalid inputs. mcp_server.py:35; capabilities.py; bob_core/file_manager.py:6-18
Output schema	PRESENT	Tools return JSON dictionaries and the Node gateway wraps responses consistently as ok/data or ok/error. mcp_server.py:31; node-server/server.js:53-111
Retry	ADDABLE	No general tool retry wrapper exists. Add retries only for transient, idempotent external calls.
Timeout	PRESENT	Colab HTTP requests and terminal/Python execution accept bounded timeouts. bob_core/colab_adapter.py:119, 138, 248; capabilities.py:267-305
Authentication	ADDABLE	Optional bearer auth is supported for Colab, but the Node gateway, Socket.IO and MCP HTTP service are unauthenticated. bob_core/colab_adapter.py:108-119; node-server/server.js
Cost	ADDABLE	No model, tool or infrastructure cost is captured. Add provider usage/cost fields to model run records.
Latency	ADDABLE	Timeouts exist, but per-tool durations and percentile metrics are not recorded. Add start/end timestamps and histograms per capability.
Security	PRESENT	Workspace path containment, allowlists, size limits and validation reduce unsafe file access. Broader API authorization remains missing. bob_core/file_manager.py:6-18, 61-82; config.py:7-53

Metric coverage: API success, retry recovery, timeout rate and argument accuracy are not measured. The tool gateway is the natural instrumentation point.

4. Retrieval-Augmented Generation

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
Chunking	NOT NEEDED	Bob builds bounded workspace context but does not implement document chunk retrieval. Current product is a coding agent using selected workspace files.
Metadata	NOT NEEDED	No retrieval index or chunk metadata exists. Add only if semantic repository/document search becomes a requirement.
Embedding	NOT NEEDED	No embedding model is used. Not required for the current staged code workflow.
Vector database	NOT NEEDED	No vector store is present. Not required at current scale/scope.
Citation	NOT NEEDED	Generated code workflow does not produce evidence-linked RAG answers. Use file/line grounding if an explanatory Q&A mode is added.
Source display	NOT NEEDED	The IDE displays selected context and diffs rather than retrieved RAG sources. Existing source/diff UI already serves the coding workflow.
Hybrid search	NOT NEEDED	No semantic retrieval pipeline exists. Only useful after adding a sizable indexed corpus.
Re-ranking	NOT NEEDED	No retrieval candidates are produced to re-rank. Not needed without RAG.

Metric coverage: Precision@K, Recall@K, Hit Rate, MRR, groundedness and citation accuracy are not applicable until a retrieval feature is introduced.

5. Structured Outputs

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
JSON output	PRESENT	MCP, model adapters and gateway responses use JSON objects. mcp_server.py:31; bob_core/colab_adapter.py:99-143
Validation	PRESENT	Plans are normalized, headers/config values are validated, and invalid inputs return understandable errors. bob_core/colab_adapter.py:13-45; bob_core/model_config.py:45-72
Pydantic model	ADDABLE	Typed functions are used, but explicit Pydantic domain models are not used for plan, run and proposal payloads. Add versioned Pydantic models at external boundaries.
Required fields	PRESENT	Plan defaults and endpoint validation ensure expected fields and basic required values are present. bob_core/colab_adapter.py:13-45; capabilities.py
Error messages	PRESENT	Validation and gateway failures return concise human-readable messages. bob_core/colab_adapter.py:99-143; node-server/server.js:77-111

Metric coverage: Schema compliance and field accuracy are not aggregated. Add contract fixtures containing valid, missing, malformed and extra-field cases.

6. Classification Evaluation

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
Confusion matrix	NOT NEEDED	Bob IDE does not expose a classification model. Not needed for the current generative coding workflow.
Accuracy	NOT NEEDED	There is no labeled classification outcome. Use task-success metrics instead.
Precision	NOT NEEDED	There is no positive-class decision to score. Not applicable.
Recall	NOT NEEDED	There is no positive-class decision to score. Not applicable.
F1	NOT NEEDED	There is no classifier label set. Not applicable.
Macro average	NOT NEEDED	No multi-class classification task exists. Not applicable.
Weighted average	NOT NEEDED	No multi-class classification task exists. Not applicable.

Metric coverage: If a future intent/risk classifier is added, introduce a labeled dataset and choose per-class precision/recall based on failure cost.

7. Agent Evaluation

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
Tool selection	ADDABLE	Tool choices are executed but not compared with labeled expected tools. Add scenario fixtures with allowed/expected tools.
Tool arguments	ADDABLE	Schemas validate shape, but semantic argument correctness is not evaluated. Add expected argument assertions and boundary cases.
Planning	ADDABLE	Plans are produced and can be reviewed, but plan quality is not scored against a rubric. Add plan completeness, relevance and step-efficiency scoring.
Memory	ADDABLE	Workflow context persists, but retention quality and unnecessary-context use are not tested. Add recall/privacy context tests.
Hallucination	ADDABLE	No automated unsupported-claim or invented-file detector is present. Validate references against workspace/tool results.
Grounding	ADDABLE	Context files are sent, but output claims are not systematically linked back to evidence. Record evidence file/line references in reviewer results.
Task success	ADDABLE	Software unit/contract tests exist, but no representative end-to-end agent task dataset exists. Add golden tasks with executable acceptance checks.
Human approval	ADDABLE	The UI confirms force actions and proposals require apply, but backend override endpoints can be invoked directly without an approval token. Enforce approval server-side with authenticated, single-use decisions.

Metric coverage: Task success, tool accuracy, average steps, loop count and completion time are absent. Instrument run records and evaluate against fixed tasks.

8. Human Evaluation

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
Correctness	ADDABLE	No formal reviewer rubric or stored human rating exists. Create a 1-5 rubric with anchored examples.
Helpfulness	ADDABLE	No human helpfulness score is captured. Add optional post-run rating and reason.
Completeness	ADDABLE	No checklist-based human completeness score is captured. Include explicit acceptance criteria per task.
Safety	ADDABLE	Proposal review reduces risk, but human safety judgments are not recorded as evaluation data. Add safety rubric and escalation reason.
Tone	ADDABLE	No tone evaluation is defined. Useful for chat responses, lower priority for code changes.
Groundedness	ADDABLE	No human evidence-grounding rubric is stored. Expose context references in the review UI.
Citation quality	NOT NEEDED	The current coding workflow does not generate formal citations. Add only for evidence-based Q&A/document modes.

Metric coverage: No Likert/rating workflow or inter-annotator agreement calculation exists. Start with a small double-reviewed sample before tracking agreement.

9. Debugging

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
Trace	ADDABLE	Run IDs link persisted stages, but tool-by-tool execution and timings do not form a complete distributed trace. Add request/trace IDs across frontend, gateway, MCP, tools and Colab.
Prompt	ADDABLE	User prompts are stored, but exact system templates and prompt versions are not recorded locally. Persist template ID, version and rendered prompt hash.
Tool logs	ADDABLE	The gateway does not persist tool name, validated arguments, result summary and duration for every call. Add redacted structured tool events.
Token logs	ADDABLE	Token input/output usage is not recorded. Capture provider usage metadata per model stage.
Error logs	ADDABLE	Failed runs store an error string, but there is no centralized typed error event with full request context. Add error taxonomy, timestamp, stage and trace ID.
Stack trace	ADDABLE	Background errors are reduced to str(exc), losing stack traces. Log tracebacks server-side while returning safe messages.
Root cause	ADDABLE	No repeatable root-cause field or workflow exists. Add component/category/root-cause fields to incidents and failed evaluations.

Metric coverage: The stored stage history is a good base, but the full input-to-output pipeline cannot yet be replayed or diagnosed from one trace.

10. Observability

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
Prompt logs	ADDABLE	No versioned, redacted prompt event stream exists. Log template ID/hash and safe metadata.
Tool logs	ADDABLE	No centralized tool-call log exists. Instrument the Node MCP call boundary and Python capability dispatcher.
Token usage	ADDABLE	No token counters are persisted. Capture per-stage input/output/cached tokens.
Latency	ADDABLE	No component duration metrics or percentiles are exported. Add histograms for gateway, MCP, Colab and tools.
Errors	ADDABLE	Errors are returned/stored, but frequency by type is not measured. Add counters by stage and error taxonomy.
Cost	ADDABLE	No cost telemetry exists. Map token/tool usage to configured unit costs.
User feedback	ADDABLE	No rating, correction or rejection signal is stored as evaluation feedback. Record apply/discard plus explicit reason, with consent.

Metric coverage: P50/P95/P99 latency, error rate and availability are not available. Health endpoints exist but are not monitored over time.

11. LLMOps

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
Prompt version	ADDABLE	No prompt registry/version identifier is present in run artifacts. Version templates and store the version on every stage.
Dataset version	ADDABLE	No versioned agent evaluation dataset exists. Create evals/ with dataset manifest and immutable case IDs.
Model version	ADDABLE	Connection settings are stored, but provider/model revision is not reliably attached to every run. Persist provider, model ID, revision and decoding settings.
Evaluation pipeline	ADDABLE	npm test covers software behavior, contracts, lint and build, but not fixed LLM quality evaluations. Add offline agent evaluations to CI with thresholds.
A/B testing	ADDABLE	No controlled prompt/model comparison or traffic split exists. Add experiment IDs and offline paired comparison first.
Rollback	ADDABLE	Git supports source rollback, but there is no automated deployment/model/prompt rollback workflow. Version releases/config and define one-command rollback.
Monitoring	ADDABLE	No continuous production quality/cost/reliability monitoring exists. Add dashboards and alerts after deployment architecture is defined.

Metric coverage: Regression, acceptance and failure rates plus deployment frequency are not calculated. The existing npm test command is a solid software-quality foundation, not an LLMOps evaluation pipeline.

12. Cloud Deployment

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
Docker	ADDABLE	No Dockerfile or container orchestration files are present. Containerize Node, Python MCP and built frontend; define health checks.
API	PRESENT	Node REST, Socket.IO and Python MCP interfaces expose core functionality. node-server/server.js; mcp_server.py
HTTPS	PRESENT	The documented Colab connection uses an HTTPS ngrok endpoint; local Node/MCP services bind over HTTP to loopback. README.md Colab/ngrok setup; local ports section
Secrets	ADDABLE	Environment variables and masking are supported, but UI-supplied bearer tokens are stored in a local JSON file rather than a secret manager. bob_core/model_config.py:74-145; data/model_config.json
Load balancer	NOT NEEDED	The current system is a single-user local/demo deployment. Add only for a multi-instance hosted service.
Autoscaling	NOT NEEDED	Colab plus local services are not a production autoscaled architecture. Needed only when hosted traffic and SLOs justify it.
Monitoring	ADDABLE	Health endpoints exist, but no cloud/application monitoring service consumes them. node-server/server.js:53-67; add uptime and resource monitoring.
Logging	ADDABLE	Runtime events are not centralized or retained as deployment logs. Add structured stdout and a log backend with redaction/retention.

Metric coverage: Requests/sec, latency, availability, cost/hour and resource utilization are not collected. Treat the current setup as a development/demo deployment.

13. Privacy, Security and Responsible AI

CHECKLIST ITEM	STATUS	PROJECT FINDING AND EVIDENCE / ACTION
Authentication	ADDABLE	The product has no user identity layer; only the Colab hop can use an optional bearer token. bob_core/colab_adapter.py:108-119; node-server/server.js
Authorization	ADDABLE	No server-side user/role permission checks protect tools or workspaces. MUTATING_TOOLS classifies writes but does not authorize them: node-server/server.js:13-39.
PII detection	ADDABLE	Inputs, selected files and model outputs are not scanned or redacted for PII. Add configurable detection before external model calls and logs.
Encryption	ADDABLE	HTTPS protects the ngrok hop, but local persisted run/context/config data is not encrypted at rest. Define threat model, encrypted secret storage and retention.
Secret management	ADDABLE	README warns against commits and environment variables are supported, but no OS/cloud secret store integration exists. README.md:976-988; bob_core/model_config.py
RBAC	NOT NEEDED	RBAC adds little to a strictly single-user local IDE. Required if Bob becomes multi-user, shared or remotely hosted.
Human approval	PRESENT	Proposals are reviewable and explicit confirmation is required for force actions in the UI. frontend/src/components/EditorArea.jsx:110-116; SourceControlPanel.jsx:48-50
Audit logs	ADDABLE	Workflow history exists, but security/data-access events are not immutable, identity-linked or retention-controlled. Add append-only redacted audit events after authentication.

Metric coverage: PII recall, unauthorized access, injection success, false refusal and data-leak rates are not tested. Add adversarial security cases before any remote/multi-user release.

14. Production Readiness

This section consolidates the original architecture, AI, evaluation, debugging, deployment, security, reliability, cost and documentation checklist.

AREA	ITEM	STATUS	ASSESSMENT / EVIDENCE
Architecture	Diagram	PRESENT	README contains a clear four-part component flow. README.md:1-13
Architecture	Components	PRESENT	README and docs define frontend, gateway, MCP and Bob Core responsibilities. README.md folder layout; docs/
Architecture	Workflow	PRESENT	The plan -> select/replan -> code -> review -> proposal -> apply flow is documented. README.md staged workflow; bob_core/model_service.py
AI	Agent	PRESENT	The autonomous goal is coding assistance inside a reviewable IDE workflow. README.md:1-31
AI	Planner	PRESENT	Planning and replanning stages are explicit. bob_core/model_service.py:75-157
AI	Tools	PRESENT	MCP capability registry exposes external actions. mcp_server.py:35-41; capabilities.py
AI	Memory	PRESENT	Workflow artifacts persist locally; retention policy is not documented. bob_core model/run/plan/proposal stores
AI	RAG	NOT NEEDED	RAG is not necessary for the current selected-file coding workflow. Add only for semantic large-corpus retrieval.
Evaluation	Dataset	ADDABLE	No representative agent-evaluation dataset exists. Add normal, edge, failure and adversarial cases.
Evaluation	Metrics	ADDABLE	No agent quality/reliability/cost scorecard exists. Tie metrics to failure cost and acceptance tests.
Evaluation	Human evaluation	ADDABLE	No rubric-backed rating workflow exists. Add anchored criteria and reviewer records.
Debugging	Logs	ADDABLE	Persisted stages are helpful but not comprehensive structured logs. Add redacted component and tool events.
Debugging	Traces	ADDABLE	Run IDs do not yet span every component/tool call with timing. Implement end-to-end trace IDs.
Debugging	Errors	ADDABLE	Errors are stored/returned but not classified consistently. Add a shared error taxonomy.
Deployment	Docker	ADDABLE	No reproducible container package exists. Add Dockerfiles and compose/deployment config.
Deployment	Cloud	PRESENT	Colab plus ngrok is selected for model compute and demo connectivity, but it is not a production platform. README.md Colab sections
Deployment	Monitoring	ADDABLE	Health checks exist without continuous monitoring/alerts. Instrument and define SLOs.
Security	Authentication	ADDABLE	No product user authentication. Add before remote hosting.
Security	Authorization	ADDABLE	No server-enforced permissions outside the LLM. Add workspace/tool policy enforcement.
Security	Secrets	ADDABLE	Secret hygiene guidance exists; managed storage does not. Use OS keychain or cloud secret manager.
Security	Encryption	ADDABLE	Partial transit protection only; no at-rest design. Document and implement encryption boundaries.

AREA	ITEM	STATUS	ASSESSMENT / EVIDENCE
Reliability	Retry	ADDABLE	No controlled transient retry policy. Use bounded exponential backoff with jitter.
Reliability	Timeout	PRESENT	Model and command calls have bounded timeouts. bob_core/colab_adapter.py; capabilities.py
Reliability	Fallback	PRESENT	Streaming falls back to synchronous execution, chat has a local fallback, and unconfigured planning returns a safe response. bob_core/colab_adapter.py:145-263; capabilities.py:1089
Reliability	Cache	ADDABLE	No result/model response cache is implemented. Add only for deterministic/safe keyed results.
Cost	Tokens	ADDABLE	Token usage is not tracked. Capture provider usage metadata.
Cost	Latency	ADDABLE	Component latency is not recorded. Add timings and percentiles.
Cost	Model routing	ADDABLE	A single configured model endpoint is used. Add routing only after quality/cost evaluation.
Cost	Cache	ADDABLE	No computation cache exists. Prioritize observability before caching.
Documentation	README	PRESENT	Comprehensive setup, operation and troubleshooting instructions exist. README.md
Documentation	API documentation	PRESENT	README and handoff docs list routes, payloads and response behavior. README.md; docs/COLAB_MCP_HANDOFF.md
Documentation	Architecture documentation	PRESENT	Multiple migration, integration and handoff documents explain decisions/components. docs/
Documentation	Demo	PRESENT	Sample projects and step-by-step demo/start/test instructions are included. workspace/sample_project; README.md
Documentation	Future work	PRESENT	Planning/handoff documents and explicit limitations identify follow-on work. docs/GIT_COLAB_INTEGRATION_PLAN.md; docs/*HANDOFF.md

Production AI Design Review

Answers are based on the present repository, with gaps stated directly.

1. Why does this need an LLM?

The LLM is justified for interpreting open-ended coding requests, planning edits across files and reviewing generated code. File permissions, path safety, validation, Git operations and apply decisions should remain deterministic.

2. What decisions are delegated?

Delegated: plan content, code generation and review judgment. Deterministic: workspace boundaries, tool schemas, timeouts, Git/proposal state, conflict checks and file writes. Strengthen this boundary with server-side authorization.

3. Five likely failure modes

Wrong/incomplete plan; missing workspace context; malformed or unsafe generated files; Colab/ngrok timeout/outage; unauthorized or accidental proposal application.

4. How will failures be detected?

Today: validation errors, HTTP exceptions, reviewer PASS/FAIL, hash conflicts and health endpoints. Missing: alerting, quality evaluations, trace coverage and security test metrics.

5. How will the system recover?

Today: replan, stream-to-sync fallback, safe failure records, discard, Git restore and explicit force handling. Add bounded retry, cancellation, incident runbooks and deployment rollback.

6. How do we know a version is better?

Not yet demonstrable for AI quality. Create a fixed versioned agent dataset and compare task success, correctness, tool accuracy, latency, tokens/cost and safety regressions.

7. How are data and secrets protected?

Current controls include local binding, path containment, optional HTTPS/bearer auth, secret masking and proposal approval. Missing: identity, authorization, PII handling, encryption at rest, managed secrets and retention policy.

8. Cost per successful task

Cannot yet be calculated because token/tool/infrastructure costs and task-success outcomes are not recorded. Store both at run level and divide total cost by accepted successful runs.

9. What breaks from 10 to 1 million users?

The local JSON stores, daemon threads, single Node/Python instances, Colab/ngrok endpoint, unauthenticated sockets, provider quotas and absent queues/monitoring become primary limits.

10. Would a customer trust it?

For a local supervised demo: reasonably, because edits are proposals and failures are visible. For remote production: not yet, until authentication, authorization, privacy controls, observability, evaluations and reproducible deployment are added.

Recommended implementation roadmap

Phase 1 - Safety boundary

Add user authentication, workspace/tool authorization, server-side approval tokens, managed secrets and stricter CORS/origin policy. Write adversarial path, permission and prompt-injection tests.

Phase 2 - Evaluation baseline

Create a versioned dataset of representative coding tasks. Run planner/tool/code/reviewer checks and executable acceptance tests. Record task success, tool selection/argument accuracy, steps, loops and completion time.

Phase 3 - Observability

Propagate trace IDs across UI, Node, MCP, tools and Colab. Add structured redacted logs, errors by taxonomy, stage/tool latency, tokens, estimated cost, feedback and monitored health checks.

Phase 4 - Reliability and LLMOps

Introduce safe retries, cancellation, idempotency keys, prompt/model versions, offline comparisons, regression thresholds, CI quality gates and rollback procedures.

Phase 5 - Deployment

Build containers, choose a hosted model/runtime, add TLS termination, persistent database/object storage, queues and backups. Add load balancing/autoscaling only when demand requires it.

Scope decisions

RAG, vector search, re-ranking, classification metrics, RBAC, load balancing and autoscaling are not inherently impossible. They are classified as not needed because the current repository is a single-user local coding IDE with selected-file context. Reclassify them as addable when product requirements introduce large-corpus retrieval, classifiers, multiple users/roles or hosted scale.